RubydDesigner / Arogundade_Glory_301908292
Type / to search
Code Issues Pull requests Agents Actions Projects Security and quality Insights Settings
Arogundade_Glory_301908292 / Arogundade_Glory_301908292_FinalSummary.md
RubydDesigner Church Management System Database
b0dcd53 · 17 hours ago History
Preview Code Blame 78 lines (42 loc) · 7.14 KB Raw

MIT 8103 Advanced Database Systems

Church Management System Database Exercise · Glory Arogundade · Student ID 301908292

Case study

Considering we were allowed to choose our own case study for this assignment, I decided to work with something I had already been thinking about for some time, a Church Management System Database. The idea came from the need to have a better way of keeping track of church records, especially members, workers, departments and activities. The database is designed for church adminstrators who handle member and department records,activity coordinators who plan programmes and monitor attendance, and workers who support the day-to-day activities of the church.

For this exercise, I created five tables: Department, Member, Worker, Activity, and Attendance. They are connected to make it easier to register members, assign workers to department, schedule departmental activities and record attendance. The database can also be used to check attendance for a particular activity, date or department.

Portfolio 1: Database Design and Modelling

I started by organising the database into five related tables. Each table has a primary key to uniquely identify its records, while foreign keys help maintain the relationships between the tables. I also included checks to make sure that certain rules are followed when data is entered. For examples, the attendance status is restricted to three options: Present, Absent or Excused.

I also used sample data to demonstrate how the database can be organised through the three stages of normalisation: 1NF, 2NF and 3NF. To test the database, I attempted 14 invalid insert operations, and all were rejected without affecting the existing valid records.

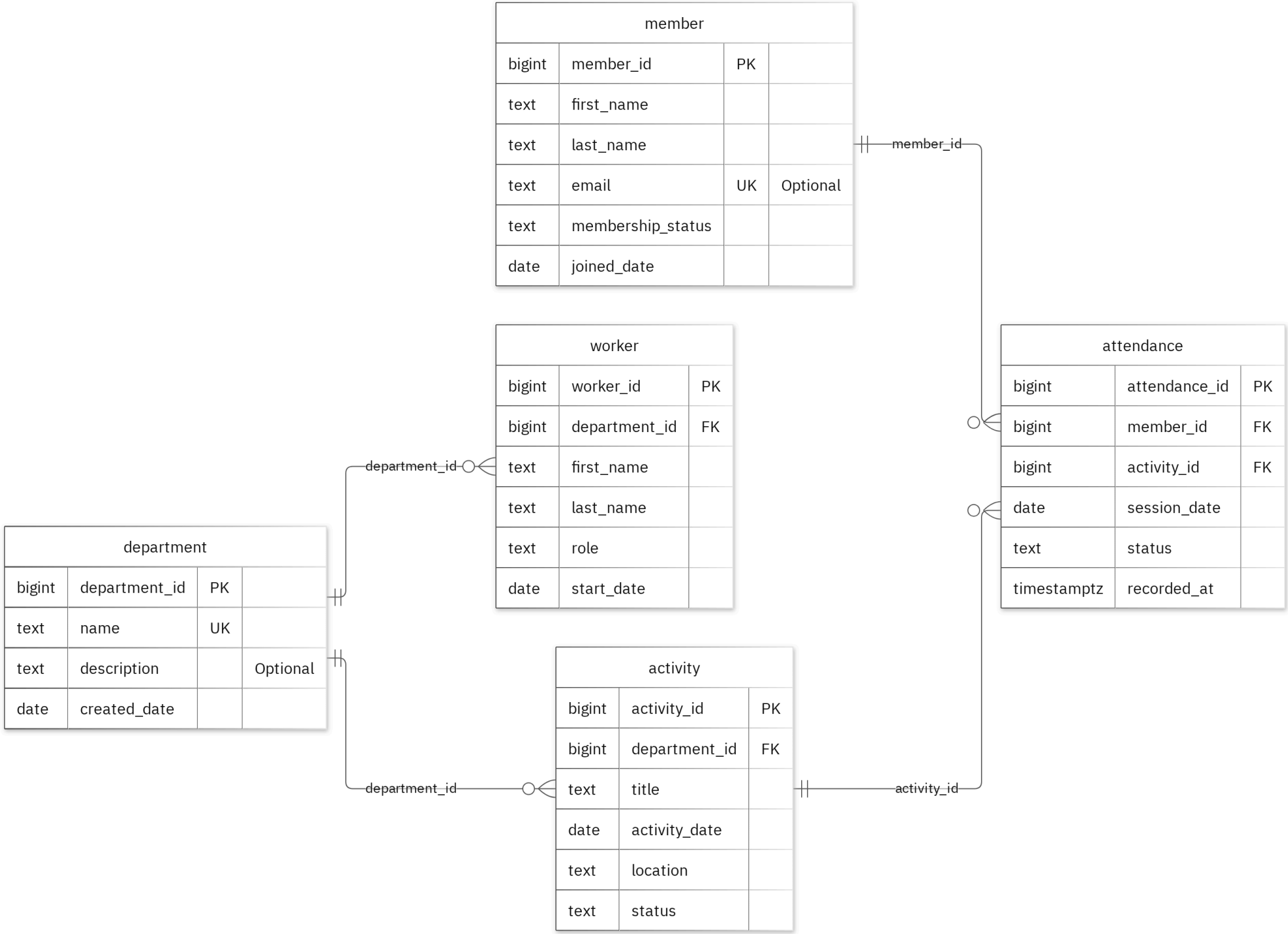

member
bigint member_id PK
text first_name
text last_name
text email UK Optional
text membership_status
date joined_date
member_id
worker
bigint worker_id PK
bigint department_id FK
text first_name
text last_name
text role
date start_date
department_id
attendance
bigint attendance_id PK
bigint member_id FK
bigint activity_id FK
date session_date
text status
timestamptz recorded_at
department
bigint department_id PK
text name UK
text description Optional
date created_date
department_id
activity
bigint activity_id PK
bigint department_id FK
text title
date activity_date
text location
text status
activity_id

Figure 1. Tables, keys and relationships. PK: primary key; FK: foreign key; UK: unique value.

Portfolio 2: Query Processing and Optimisation

I also ran three SQL queries to test how the database works with the sample records. One of the queries used a JOIN to retrieve related information from different tables, and it returned one record showing Alex's attendance at the community gathering.

For the index performance test, I used a seperate set of 20,000 records to compare how quickly data could be retrieved before and after creating an index. The median lookup time reduced from 0.941ms to 0.167ms after the index was added. Both tests returned the same records, and the execution plan confirmed that the new index was being used. These results are based on the particular lookup and test environment I used.

Portfolio 3: Transactions and Concurrency

For Transactions and Concurrency, I used a temporary counter in the attendance table to test how database transactions work. I carried out tests usiNg COMMIT,ROLLBACK and READ COMMITTED isolation. During the test, an update that had not yet been committed could not be seen by the other session. When i used ROLLBACK, the counter returned to its previous saved value.

I also tested what happens when two sessions try to update the same record at the same time. Without locking, both increments resulted in a final value of 2 instead of 3. However, after using SELECT FOR UPDATE, session B had to wait for session A to finish before making its update, and the final counter value was correctly recorded as 3. The temporary counter was only used for these exercises and did not change the original business fields of the case study.

Portfolio 4: NoSQL and Advanced Data Models

Here I used church_activity_events to store event information for the church management system. MongoDB inserted 2 sample events and ran 4 queries, all of which returned the expected results.

I also tested the validation rules by trying to insert an event with an invalid date format and another with an event ID that already existed. Both were rejected. One thing i demonstrated was how related information can be stored together within an event using embedded

snapshots, while the details field can contain different information depending on the type of event. The events used were fictional, and PostgreSQL remained responsible for enforcing the original relational database rules.

Repository and evidence

https://github.com/RubydDesigner/Arogundade_Glory_301908292

The portfolio folders contain the SQL and JavaScript, explanations, query plans, result tables and screenshots.

Portfolio 5: Distributed and Cloud Database Exercise

Here i discussed how the church database could be made more reliable by using three Mondo DB replica members across seperate failure zones. I covered how primary elections work, majority acknowledgemet,delays in secondary updates and the advatages and disadvatages of sharding for the church_activity_events collection.

I also explained that replication helps keep the database available when one member becomes unavailable, while backups are still needed to recover lost data.PostgreSQL would also need its own plan for availability and recovery. This is the permitted discussion option, so i did not set up an actual replica set or carry out a live failover test.

Individual reflection

While working on this project, I realised the importance of keeping related information properly organised in a database. I used seperate tables for Department, Member, Worker, Activity and Attendance so that the information could be managed without unnneccessary repetition. I also added rules to ensure that the attendance could only be recorded as Present,Absent or Excused, and tested the database to make sure these rules and the links between records were working correctly.

One thing i learnt from the two-session exercise was how easily database records can be affected when two users try to update the same information at the same time. Both sessions initially read the same value, which caused an issue with the final result. Using row locking helped prevent this and ensured that both updates were properly recorded.

The Index comparison also helped me understand that getting the correct result is not the only thing that matters. It is equally important to consider how the database retrieves the information and whether it is doing so efficiently.

AI-use declaration

I used GitHub Copilot and the portfolio generator to help prepare the database model, and scripts. The generator ran the PostgreSQL and MongoDB exercises, checked their outputs and captured the result images. I remain responsible for checking and explaining the submitted work.

References

- PostgreSQL: constraints
- PostgreSQL: EXPLAIN
- PostgreSQL: isolation
- PostgreSQL: row locks
- MongoDB: data modelling
- MongoDB: replication
- MongoDB: sharding
- Mermaid: ER diagrams